Algorithm 1: Index Construction of HNSW

Input: a vector dataset S, hyper-parameters C and R ($R \leq C$)
Output: HNSW index built on S

1 $V \leftarrow \emptyset, E \leftarrow \emptyset$; ▷ start from a empty graph
2 **for** *each* x *in* S **do** ▷ insert all vectors in S
3 $l_{max} \leftarrow x$'s max layer; ▷ exponential decaying distribution
4 **for** *each* $l \in \{l_{max}, \cdots, 0\}$ **do** ▷ insert x at layer l
5 $C(x) \leftarrow$ top-C candidates; ▷ greedy search
6 $N(x) \leftarrow \leq R$ neighbors from $C(x)$; ▷ heuristic strategy
7 add x to $N(y)$ for each $y \in N(x)$; ▷ reverse edge
8 $V \leftarrow V \cup \{x\}, E \leftarrow E \cup N(x)$
9 **return** $G = (V, E)$

maximum layer l_{max} is randomly selected following an exponentially decaying distribution (line 3). The vector is inserted into layers from l_{max} to 0, with the same procedure applied at each layer (lines 4-7). At layer l, x is treated as a query point, and a greedy search on the current graph at layer l identifies the top-C nearest vertices as candidates (line 5). During the search, a candidate set $C(x)$ of size C is maintained, with vertices ordered in ascending distance to x, and the maximum distance is denoted as T. The search iteratively visits a vertex's neighbors, calculates their distances to x, and updates $C(x)$ with neighbors that have smaller distances ($< T$). To obtain the final neighbors, a heuristic edge selection strategy prevents clustering among neighbors (line 6). For example, for candidate v and any candidate u where $\delta(u, x) < \delta(v, x)$, if $\delta(u, v) < \delta(v, x)$, v is excluded from $N(x)$. After determining x's final neighbors, x is added to the neighbor list $N(y)$ for each $y \in N(x)$ (line 7). If $N(y)$ exceeds R, the heuristic edge selection strategy prunes excess neighbors, ensuring that the neighbor count remains under R.

The construction process of HNSW involves two main steps: *Candidate Acquisition* (CA) and *Neighbor Selection* (NS). The CA step identifies the top-C nearest vertices for an inserted vector using a greedy search strategy. This involves visiting a vertex's neighbor list and computing distances to the inserted vector, requiring random memory access to fetch a neighbor's vector data, stored separately from neighbor IDs due to the large vector size. In the NS step, the final R neighbors are selected from the candidate set using a heuristic strategy. Here, distances among candidates are computed, also requiring random access to their vector data. In both CA and NS, distance calculations are used solely for comparison. In CA, distances are compared to the largest distance in the candidate set to determine if a visiting vertex can replace the farthest candidate. In NS, distances among candidates and between candidates and the inserted vector are compared to decide the inclusion of a candidate in the final neighbor set. HNSW currently performs all distance computations on full-precision vectors. Additionally, to utilize SIMD acceleration, a distance computation requires hundreds of read operations to load vector segments into a 128-bit register due to a large vector size, often spanning thousands of bytes.

Example 1. In Figure 2, we illustrate the index construction process of HNSW at the base layer, exemplified by inserting v_{18} with $C = 4$ and $R = 2$. In the CA step, v_{18} is treated as a query point, and a greedy search from v_0 is initiated. After examining v_0's neighbors, the candidate set $C(v_{18})$ is $\{v_5, v_9, v_0, v_{13}\}$, ordered by proximity

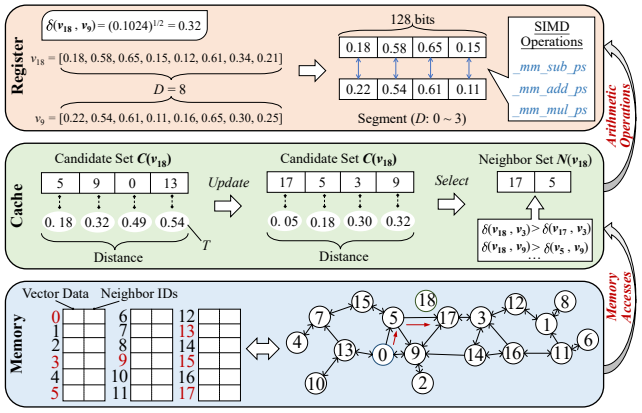


Figure 2: Illustrating memory accesses and arithmetic operations during the HNSW construction process (base layer).

to v_{18}. The current distance threshold T is $\delta(v_{13}, v_{18}) = 0.54$. The nearest vertex, v_5, is then visited, prompting an exploration of its neighbors to update $C(v_{18})$. Upon calculating $\delta(v_{15}, v_{18})$ and comparing it with T, v_{13} is replaced by v_{15} as $\delta(v_{15}, v_{18}) < T$, and T is updated to $\delta(v_0, v_{18}) = 0.49$. This process continues, resulting in the replacement of v_0 and v_{15} with v_{17} and v_3, respectively. At the end of CA, $C(v_{18})$ comprises $\{v_{17}, v_5, v_3, v_9\}$. The vertices v_0, v_3, v_5, v_9, v_{13}, v_{15}, and v_{17} are traversed in this phase. As shown in the graph index's memory layout (lower left), these vertices (highlighted in red) exhibit a random distribution, meaning numerous random memory accesses. In the NS step, v_{17} is initially chosen, leading to the removal of v_3 and v_9 from $C(v_{18})$ based on the conditions $\delta(v_{17}, v_3) < \delta(v_3, v_{18})$ and $\delta(v_{17}, v_9) < \delta(v_9, v_{18})$. The remaining vertex, v_5, is added to the neighbor list, which reaches the limit $R = 2$, yielding $\{v_{17}, v_5\}$. v_{18} is then appended to the neighbor lists of v_{17} and v_5, while ensuring that their neighbor counts remain below R through the heuristic strategy. In practice, $C(v_{18})$ can include thousands of candidates, making full vector caching impractical and necessitating many random memory accesses for distance computations during the NS stage. To leverage SIMD instructions, a vector is divided into multiple segments, each comprising four dimensions suitable for loading into a 128-bit register. While SIMD enhances processing speed, the necessity of numerous register load operations to sequentially fetch each vector segment leads to reduced SIMD utilization efficiency.

Our analysis reveals that the primary inefficiency in HNSW index construction lies in the current distance computation process, consuming over 90% of indexing time (Figure 1). This process suffers from numerous random memory accesses and suboptimal SIMD utilization, misaligning with modern CPU architecture. Thus, optimizing distance computation to better exploit modern CPU capabilities is essential for accelerating HNSW construction.

3 METHODOLOGY

In this section, we theoretically analyze how distance comparisons in HNSW construction can be effectively executed using compact vector codes. Based on this, we integrate three common vector compression methods—Product Quantization (PQ) [67], Scalar Quantization (SQ) [29], and Principal Component Analysis (PCA)

[69]—into HNSW. While many variants of these methods exist [51, 52, 57, 59, 99, 105, 109, 122], we exclude them due to their complexity compared to the original methods, which complicates deployment in HNSW. We focus on these three methods for their lightweight nature and alignment with our goal of accelerating index construction. Drawing from the integration of these methods in HNSW construction, we design a novel compact coding strategy and optimize the memory layout for HNSW construction, enhancing memory access efficiency and SIMD operations.

3.1 Theoretical Analysis

Recall that distance computation during HNSW construction is a major bottleneck, with one distance value primarily compared against another. We denote distance comparisons for updating the candidate set as DC1, and for selecting final neighbors as DC2. Here, we demonstrate that DC1 and DC2 can be unified into a generalized framework and analyze how these comparisons can be effectively performed using compact vector codes.

Lemma 1. *Given any three vertices $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ in D-dimensional Euclidean space $\mathbb{R}^D$, the comparison between $\delta(\boldsymbol{u},\boldsymbol{v})$ and $\delta(\boldsymbol{u},\boldsymbol{w})$ is:*
- *$\delta(\boldsymbol{u},\boldsymbol{v}) < \delta(\boldsymbol{u},\boldsymbol{w})$ if and only if $\boldsymbol{e}\cdot\boldsymbol{u} - b < 0$;*
- *$\delta(\boldsymbol{u},\boldsymbol{v}) > \delta(\boldsymbol{u},\boldsymbol{w})$ if and only if $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$;*
- *$\delta(\boldsymbol{u},\boldsymbol{v}) = \delta(\boldsymbol{u},\boldsymbol{w})$ if and only if $\boldsymbol{e}\cdot\boldsymbol{u} - b = 0$;*

where $\boldsymbol{e}\cdot\boldsymbol{u} - b = 0$ defines the perpendicular bisector hyperplane between $\boldsymbol{v}$ and $\boldsymbol{w}$, with $\boldsymbol{e} = \boldsymbol{w} - \boldsymbol{v}$ and $b = \frac{\|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2}{2}$.

Proof. To show $\delta(\boldsymbol{u},\boldsymbol{v}) = \delta(\boldsymbol{u},\boldsymbol{w})$, we square both sides to eliminate square roots, giving $\|\boldsymbol{u}-\boldsymbol{v}\|^2 = \|\boldsymbol{u}-\boldsymbol{w}\|^2$. Expanding and simplifying, we get $2\boldsymbol{u}\cdot(\boldsymbol{w}-\boldsymbol{v}) = \|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2$. Defining $\boldsymbol{e} = \boldsymbol{w}-\boldsymbol{v}$ and $b = \frac{\|\boldsymbol{w}\|^2-\|\boldsymbol{v}\|^2}{2}$, this simplifies to $\boldsymbol{e}\cdot\boldsymbol{u} = b$, the equation of a hyperplane in $\mathbb{R}^D$. For $\boldsymbol{u}$ to lie on this hyperplane, it must satisfy $\boldsymbol{e}\cdot\boldsymbol{u} - b = 0$. Similarly, for $\delta(\boldsymbol{u},\boldsymbol{v}) > \delta(\boldsymbol{u},\boldsymbol{w})$, squaring both sides gives $\|\boldsymbol{u}-\boldsymbol{v}\|^2 > \|\boldsymbol{u}-\boldsymbol{w}\|^2$, leading to $2\boldsymbol{u}\cdot(\boldsymbol{w}-\boldsymbol{v}) > \|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2$, and thus $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$. For $\delta(\boldsymbol{u},\boldsymbol{v}) < \delta(\boldsymbol{u},\boldsymbol{w})$, a similar process gives $\boldsymbol{e}\cdot\boldsymbol{u} - b < 0$. □

In DC1, with $\boldsymbol{u}$ as the newly inserted vector and $\boldsymbol{v}$ as the farthest vertex from $\boldsymbol{u}$ in the candidate set $\boldsymbol{C(u)}$, we update $\boldsymbol{C(u)}$ by including $\boldsymbol{w}$ if $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$. In DC2, with $\boldsymbol{v}$ as the newly inserted vector and $\boldsymbol{w}$ as a selected neighbor in $\boldsymbol{N(v)}$, we exclude a candidate $\boldsymbol{u}$ from consideration as a neighbor if $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$. Using $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$ as a criterion, both DC1 and DC2 can be correctly executed.

Theorem 1 ([29]). *For three vertices $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ in D-dimensional Euclidean space $\mathbb{R}^D$, with compact vector codes $\boldsymbol{u'}$, $\boldsymbol{v'}$, and $\boldsymbol{w'}$, the condition $\delta(\boldsymbol{u},\boldsymbol{v}) > \delta(\boldsymbol{u},\boldsymbol{w})$ is equivalent to $\delta(\boldsymbol{u'},\boldsymbol{v'}) > \delta(\boldsymbol{u'},\boldsymbol{w'})$ when $|\boldsymbol{e}\cdot\boldsymbol{u} - b| \geq |E|$. The quantity E is defined as*

$$\begin{aligned} E =&(E_{\boldsymbol{w}} - E_{\boldsymbol{v}})\cdot\boldsymbol{u} + (\boldsymbol{w}-\boldsymbol{v})\cdot E_{\boldsymbol{u}} + E_{\boldsymbol{v}}\cdot E_{\boldsymbol{u}} - E_{\boldsymbol{w}}\cdot E_{\boldsymbol{u}} \\ &+ \frac{1}{2}||E_{\boldsymbol{w}}||^2 - \frac{1}{2}||E_{\boldsymbol{v}}||^2 + \boldsymbol{v}\cdot E_{\boldsymbol{v}} - \boldsymbol{w}\cdot E_{\boldsymbol{w}} \end{aligned} \tag{1}$$

where $E_{\boldsymbol{u}}$, $E_{\boldsymbol{v}}$, and $E_{\boldsymbol{w}}$ are the error vectors corresponding to $\boldsymbol{u}$, $\boldsymbol{v}$, and $\boldsymbol{w}$, respectively.

Proof. Based on Lemma 1, the condition $\delta(\boldsymbol{u'},\boldsymbol{v'}) > \delta(\boldsymbol{u'},\boldsymbol{w'})$ is equivalent to $\boldsymbol{e'}\cdot\boldsymbol{u'} - b' > 0$, where $\boldsymbol{e'} = \boldsymbol{w'} - \boldsymbol{v'}$ and $b' = \frac{\|\boldsymbol{w'}\|^2-\|\boldsymbol{v'}\|^2}{2}$. To demonstrate that $\delta(\boldsymbol{u},\boldsymbol{v}) > \delta(\boldsymbol{u},\boldsymbol{w})$ is equivalent to $\delta(\boldsymbol{u'},\boldsymbol{v'}) > \delta(\boldsymbol{u'},\boldsymbol{w'})$, it suffices to prove that $\boldsymbol{e}\cdot\boldsymbol{u} - b$ and $\boldsymbol{e'}\cdot\boldsymbol{u'} - b'$ share the same sign. Substituting $\boldsymbol{e'}$ and b' into $\boldsymbol{e'}\cdot\boldsymbol{u'} - b'$, we obtain:

$$\boldsymbol{e'}\cdot\boldsymbol{u'} - b' = (\boldsymbol{w'}-\boldsymbol{v'})\cdot\boldsymbol{u'} - \frac{\|\boldsymbol{w'}\|^2 - \|\boldsymbol{v'}\|^2}{2} \tag{2}$$

For vector $\boldsymbol{u}$, let $E_{\boldsymbol{u}}$ denote the compression error of $\boldsymbol{u}$, defined as

$$E_{\boldsymbol{u}} = \boldsymbol{u} - \boldsymbol{u'} \tag{3}$$

Substituting this into the previous equation, we derive:

$$\begin{aligned} \boldsymbol{e'}\cdot\boldsymbol{u'} - b' =&((\boldsymbol{w} - E_{\boldsymbol{w}}) - (\boldsymbol{v} - E_{\boldsymbol{v}}))\cdot(\boldsymbol{u} - E_{\boldsymbol{u}}) \\ &- \frac{\|\boldsymbol{w} - E_{\boldsymbol{w}}\|^2 - \|\boldsymbol{v} - E_{\boldsymbol{v}}\|^2}{2} \end{aligned} \tag{4}$$

Expanding and simplifying, we group terms involving compression errors:

$$\begin{aligned} E =&(E_{\boldsymbol{w}} - E_{\boldsymbol{v}})\cdot\boldsymbol{u} + (\boldsymbol{w}-\boldsymbol{v})\cdot E_{\boldsymbol{u}} + E_{\boldsymbol{v}}\cdot E_{\boldsymbol{u}} - E_{\boldsymbol{w}}\cdot E_{\boldsymbol{u}} \\ &+ \frac{1}{2}||E_{\boldsymbol{w}}||^2 - \frac{1}{2}||E_{\boldsymbol{v}}||^2 + \boldsymbol{v}\cdot E_{\boldsymbol{v}} - \boldsymbol{w}\cdot E_{\boldsymbol{w}} \end{aligned} \tag{5}$$

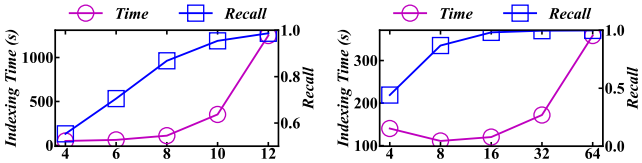
Thus, the equation simplifies to:

$$\begin{aligned} \boldsymbol{e'}\cdot\boldsymbol{u'} - b' &= (\boldsymbol{w}-\boldsymbol{v})\cdot\boldsymbol{u} - \frac{1}{2}(||\boldsymbol{w}||^2 - ||\boldsymbol{v}||^2) - E \\ &= \boldsymbol{e}\cdot\boldsymbol{u} - b - E \end{aligned} \tag{6}$$

Therefore, when $|\boldsymbol{e}\cdot\boldsymbol{u} - b| \geq |E|$, the sign of $\boldsymbol{e'}\cdot\boldsymbol{u'} - b'$ matches that of $\boldsymbol{e}\cdot\boldsymbol{u} - b$. Specifically, $\boldsymbol{e'}\cdot\boldsymbol{u'} - b' > 0$ if and only if $\boldsymbol{e}\cdot\boldsymbol{u} - b > 0$, and similarly, $\boldsymbol{e'}\cdot\boldsymbol{u'} - b' < 0$ if and only if $\boldsymbol{e}\cdot\boldsymbol{u} - b < 0$. □

Theorem 1 elucidates how distance comparison is preserved despite compression error. By adjusting the parameters of compact coding (thus affecting E), the condition $|\boldsymbol{e}\cdot\boldsymbol{u} - b| \geq |E|$ can always be satisfied. Consequently, an appropriate compression error maintains accurate distance comparison while reducing vector data size, enhancing memory access and SIMD operations. Hence, integrating vector compression techniques into the HNSW construction process is a promising avenue for accelerating index construction. Given a vector $\boldsymbol{u}$ and its compact vector code $\boldsymbol{u'}$, the error vector is computed as $E_{\boldsymbol{u}} = \boldsymbol{u} - \boldsymbol{u'}$. Here, $\boldsymbol{u'}$ refers to the vector derived from the compact vector code, not the vector code itself; this derived vector retains the same dimensionality as $\boldsymbol{u}$. For different compact coding methods, the derived vectors have different interpretations, and thus $E_{\boldsymbol{u}}$ is computed in distinct ways (see Section 3.2).

In a compact coding method, adjustable parameters control the compression error. Specifically, a random sample of vectors (e.g., 10,000) is drawn from the dataset, and each vector's top-100 nearest neighbors are determined, forming a triple $(\boldsymbol{u},\boldsymbol{v},\boldsymbol{w})$ from the vector and its two nearest neighbors. A batch of such triples is then constructed, followed by the generation of compact vector codes $\boldsymbol{u'}$, $\boldsymbol{v'}$, and $\boldsymbol{w'}$ for specific parameters. Next, the quantities $|\boldsymbol{e}\cdot\boldsymbol{u} - b|$ and $|E|$ are computed for each triple. By tuning the parameters, one can maximize the proportion of triples that satisfy $|\boldsymbol{e}\cdot\boldsymbol{u} - b| \geq |E|$ while minimizing the vector size. Finally, the chosen parameters are evaluated by integrating the compact coding method into HNSW.



(a) LAION-1M (M_{PQ} = 8) (b) LAION-1M (L_{PQ} = 8)

Figure 3: Effect of parameters on HNSW-PQ.

3.2 Baseline Solutions

3.2.1 ***Deploying PQ in HNSW.*** Product Quantization (PQ) [67] splits high-dimensional vectors into subvectors that are independently compressed. For each subspace, a codebook of centroids is generated, and during encoding, the ID of the nearest centroid is selected to create a compact representation. The trade-off between distance comparison accuracy and compression error is managed by adjusting the code length (L_{PQ}) in each subspace and the number of subspaces (M_{PQ}). For example, with M_{PQ} = 8 and L_{PQ} = 4, a vector is represented by 8 codewords, each storing a 4-bit centroid ID in the subspace. For a vector $\boldsymbol{u}$ and its compact code $\boldsymbol{u'}$, the derived vector is formed by concatenating $\boldsymbol{u}$'s nearest centroid vectors (identified by $\boldsymbol{u'}$) from all subspaces. The error vector $E_{\boldsymbol{u}}$ is then computed as the difference between $\boldsymbol{u}$ and this derived vector.

To incorporate PQ into HNSW, we preprocess high-dimensional vectors to create codebooks. Each inserted vector is encoded using these codebooks, and a distance table is computed between the vector and subspace centroids (Asymmetric Distance Computation, ADC [67]). In the Candidate Acquisition (CA) stage, distances to visited vertices are efficiently computed by scanning this table. In the Neighbor Selection (NS) stage, the table cannot compute candidate distances. Thus, centroids are located based on compact PQ codes, and inter-centroid distances represent candidate distances (Symmetric Distance Computation, SDC [67]). Theorem 1 suggests that optimal values of L_{PQ} and M_{PQ} balance distance comparison accuracy with construction efficiency. We denote the HNSW constructed via this pipeline as HNSW-PQ, which replaces original vectors with PQ codes and employs ADC and SDC for efficient distance computation during index construction.

We evaluate HNSW-PQ under different L_{PQ} and M_{PQ} configurations, as shown in Figure 3. The results indicate that both parameters significantly influence indexing time and search performance. As L_{PQ} grows, more clusters are formed in each subspace, leading to longer codebook generation and vector encoding times, thus increasing indexing time. Notably, indexing time initially decreases before rising with M_{PQ}. This occurs because a small M_{PQ} causes high subspace dimensionality, increasing codebook generation time. Conversely, a larger M_{PQ} also raises indexing time due to more subspaces. Regarding index quality, both higher L_{PQ} and M_{PQ} improve recall, aligning with the theoretical analysis that lower compression errors yield more accurate distance comparisons.

3.2.2 ***Deploying SQ in HNSW.*** Scalar Quantization (SQ) [29, 36] compresses each dimension of vectors independently. It divides data ranges into intervals, assigning each interval an integer value. Floating-point values within an interval are approximated by that integer value, thus reducing vector size by lowering the required bits (L_{SQ}). For example, with L_{SQ} = 8, each dimension is represented by an integer (0~255). Before distance computation, these integers are converted back into floating-point numbers to form the decoded vector, which is lossy. The error vector $E_{\boldsymbol{u}}$ is obtained by subtracting the decoded vector from $\boldsymbol{u}$.

To accelerate HNSW construction via SQ, we preprocess high-dimensional vectors to identify the maximum and minimum values per dimension. We calculate the interval for each dimension by subtracting the minimum from the maximum. During index construction, each inserted vector is encoded using these precomputed intervals, producing compact SQ codes for distance computations. As per Theorem 1, the optimal L_{SQ} balances distance comparison accuracy with construction efficiency. The resulting HNSW, termed HNSW-SQ, replaces original vector data with compact SQ codes, enabling efficient distance calculations.

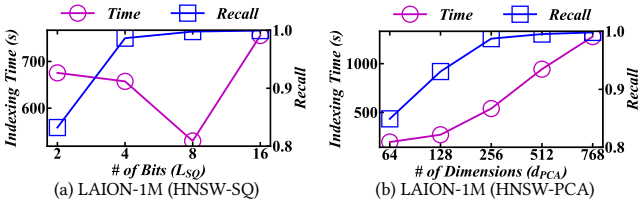
In Figure 4(a), we assess HNSW-SQ across varying L_{SQ} values. The results show that indexing time first decreases and then increases as L_{SQ} grows. This trend arises from the lack of specialized data types for 2 and 4 bits, necessitating the use of *uint8* type, which reduces operational efficiency. In contrast, 8 bits, while larger, aligns well with *uint8*, achieving an optimal balance between compression error and operational efficiency, thereby reducing indexing time. While 16 bits can be represented with *uint16*, the larger vector size increases indexing time. Notably, search accuracy consistently improves with higher L_{SQ}, supporting the theoretical analysis.

3.2.3 ***Deploying PCA in HNSW.*** Principal Component Analysis (PCA) [69] reduces dimensionality by projecting high-dimensional vectors into a lower-dimensional space. It applies an orthogonal transformation matrix to a vector $\boldsymbol{u}$, preserving its norm while prioritizing significant components in lower dimensions. The compact vector code $\boldsymbol{u'}$ is formed by retaining the first d_{PCA} dimensions, substantially reducing storage requirements. The error vector $E_{\boldsymbol{u}}$ is calculated by subtracting the dimension-reduced vector from the transformed vector of $\boldsymbol{u}$, with the dimension-reduced vector zero-padded to match dimensionality.

To expedite HNSW construction via PCA, we preprocess high-dimensional vectors by deriving an orthogonal projection matrix through eigenvalue decomposition of the covariance matrix. Each inserted vector is then transformed into a low-dimensional space using this matrix, with principal components serving as compact PCA codes. Distance computations are performed directly on these compact representations. Theorem 1 indicates that an optimal d_{PCA} balances distance comparison accuracy and construction efficiency. The resulting HNSW, designated as HNSW-PCA, substitutes the original vectors with compact PCA representations, thereby facilitating more efficient distance computations.

Figure 4(b) shows the evaluation of HNSW-PCA across various d_{PCA} configurations. We find that indexing time increases with higher d_{PCA}, while search accuracy improves, aligning with the theoretical analysis. Optimal trade-off between indexing time and search performance occurs with d_{PCA} ranging from 256 to 512. Notably, d_{PCA} reaches 420 when 90% cumulative variance is achieved on the LAION dataset. In the experiments, d_{PCA} will be set to ensure at least 90% cumulative variance.

3.2.4 ***Lessons Learned.*** Theoretical and empirical analyses reveal that index construction can be accelerated using compact coding techniques, and construction efficiency and index quality can be well balanced by adjusting the compression error. An appropriate



(a) LAION-1M (HNSW-SQ) (b) LAION-1M (HNSW-PCA)

Figure 4: Effect of parameters on HNSW-SQ and HNSW-PCA.

compression error reduces vector data size, improving memory access and SIMD operation efficiency while preserving accurate distance comparisons. Following this insight, optimized variants [36, 51, 52, 57, 59, 99, 105, 109, 122] of PQ, SQ, and PCA may be integrated into HNSW to further speed up index construction. Note that these variants must avoid excessive processing overhead to maintain the balance between construction efficiency and index quality. Nonetheless, the core mechanism of these methods—vector size reduction—does not align well with HNSW's construction characteristics on modern CPUs. Regardless of whether HNSW-PQ, HNSW-SQ, or HNSW-PCA is used, the random memory access pattern during index construction remains unchanged. In terms of arithmetic operations, HNSW-PQ's distance table lookup is unable to fully utilize SIMD operations, as each computation is executed individually [51]. While HNSW-SQ and HNSW-PCA reduce register loads by computing directly on compressed vectors, they still process distance computations sequentially, underutilizing SIMD's advantages [30]. We highlight that current variants of PQ, SQ, and PCA do not fundamentally resolve these limitations.

3.3 The Flash Method

In Section 3.2, we demonstrate that incorporating existing compact coding methods into HNSW construction can enhance index construction speed. However, these methods either offer limited gains in indexing efficiency or degrade search performance. This is mainly because they are unable to effectively reduce random memory accesses and fully exploit the advantages of SIMD instructions. To overcome these limitations, we propose a specific compact coding strategy alongside memory layout optimization for the construction process of HNSW, named Flash, which minimizes random memory accesses while maximizing SIMD utilization.

3.3.1 ***Design Overview.*** Noting that HNSW-PQ strikes an optimal balance between construction efficiency and search performance, with PQ offering flexibility through two adjustable parameters that affect the compression error. We leverage the core principles of PQ to devise Flash, tailored to the HNSW construction process on modern CPUs. Figure 5 provides an overview of the coding pipeline and data layout used by Flash.

Given that a CPU core has multiple SIMD registers, Flash partitions the high-dimensional space into subspaces, enabling parallel processing of subspaces in these registers. To accommodate the distinct distance computations in Candidate Acquisition (CA) and Neighbor Selection (NS) stages, it introduces Asymmetric Distance Tables (ADT) for the CA stage, residing in SIMD registers, and Symmetric Distance Tables (SDT) for the NS stage, located in cache. It optimizes the bit counts allocated for each codeword (i.e., encoded vector) and applies SQ to compress the precomputed ADT to fit within a SIMD register, thereby reducing register loads. Since high-dimensional vectors often exhibit uneven variances across dimensions, directly encoding original subspaces may result in suboptimal bit utilization. To address this, Flash utilizes PCA to extract the principal components of vectors, generating subspaces based on these components, enhancing bit utilization within the SIMD register's constraints. When organizing neighbor lists, neighbor IDs are grouped with corresponding codewords to minimize random memory accesses. Rather than sequentially storing all codewords for a neighbor, Flash gathers the codewords of a batch of neighbors within a specific subspace. The batch size matches the number of centroids per subspace. This alignment allows a register load to fetch an entire batch, enabling partial distances to be obtained in parallel using SIMD shuffle operations since the ADT for a subspace is fully contained within a SIMD register.

Flash enhances HNSW construction by optimizing memory layout and SIMD operations. Two hyperparameters—the number of subspaces (M_F) and the dimension of the principal components (d_F)—can be adjusted to balance construction efficiency and search performance (Theorem 1). These adjustments do not affect Flash's memory access pattern or SIMD optimizations. We highlight that existing SIMD optimizations for PQ, as discussed in [30, 31], target linear scans or inverted file (IVF) structures, where vector batches are naturally grouped, rather than graph index settings [51].

3.3.2 ***Extracting Principal Components.*** To align with SIMD register constraints, each subspace has a limited bit representation (e.g., 4 bits). Encoding on principal components optimally utilizes these bits, reducing quantization error. Thus, Flash employs PCA to project high-dimensional vectors by rearranging dimensions in descending order of variance. This approach preserves essential low-dimensional principal components by selecting the first d_F dimensions, encapsulating the most significant features. Given a dataset $\boldsymbol{S}$ of n vectors, each with D dimensions, the mean vector is $\bar{\boldsymbol{u}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{u}_i$. Data is centered by subtracting $\bar{\boldsymbol{u}}$ from each vector, producing the centered matrix $\dot{\boldsymbol{S}} = \boldsymbol{S} - \mathbf{1}\cdot\bar{\boldsymbol{u}}^\top$, where $\mathbf{1}$ is a unit vector. The covariance matrix $\Sigma = \frac{1}{n}(\dot{\boldsymbol{S}}^\top\dot{\boldsymbol{S}})$ is then computed, and its eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_D$ and eigenvectors $\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_D$ are extracted. For a given variance fraction α, the function $f(d) = \frac{\sum_{i=1}^{d}\lambda_i}{\sum_{i=1}^{D}\lambda_i}$ identifies the smallest d_F such that $f(d_F) \geq \alpha$, defining the principal component space of dimension d_F. The principal components of each vector are established based on the basis $\boldsymbol{A}_{1:d_F} = (\boldsymbol{a}_1\ \boldsymbol{a}_2\ \cdots\ \boldsymbol{a}_{d_F})$, yielding the reduced-dimensional data $\widetilde{\boldsymbol{S}}$:

$$\widetilde{\boldsymbol{S}} = \{\tilde{\boldsymbol{u}}_i \mid \tilde{\boldsymbol{u}}_i = \mathbf{A}_{1:d_F}^\top \boldsymbol{u}_i,\ \text{for } i = 1, \ldots, n\} \tag{7}$$

3.3.3 ***Subspace Division and Distance Table Compression.*** Flash decomposes each vector's principal components $\tilde{\boldsymbol{u}}$ into M_F disjoint subvectors, denoted as $\tilde{\boldsymbol{u}} = [\tilde{\boldsymbol{u}}_1, \tilde{\boldsymbol{u}}_2, \ldots, \tilde{\boldsymbol{u}}_{M_F}]$, where each $\tilde{\boldsymbol{u}}_i$ resides in a specific subspace of the principal component domain. Similar to PQ, a codebook $\boldsymbol{C}_i = \{\boldsymbol{c}_{i1}, \boldsymbol{c}_{i2}, \ldots, \boldsymbol{c}_{iK}\}$ is created for each subspace, with $\boldsymbol{c}_{ij}$ denoting a codeword (i.e., a centroid ID) in the i-th subspace, and K representing the number of centroids. The i-th subvector $\tilde{\boldsymbol{u}}_i$ is quantized by identifying the closest centroid in the codebook $\boldsymbol{C}_i$ of the i-th subspace:

$$\psi(\tilde{\boldsymbol{u}}_i) = \arg\min_{\boldsymbol{c}_{ij} \in \boldsymbol{C}_i} \delta(\tilde{\boldsymbol{u}}_i, \boldsymbol{c}_{ij}) \tag{8}$$

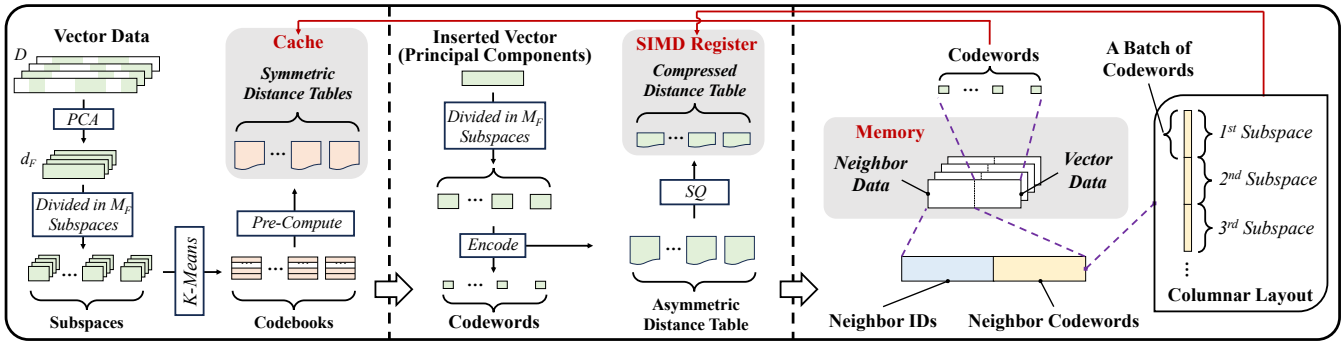


Figure 5: Illustrating the coding pipeline and data layout of Flash.

where $\psi(\tilde{\boldsymbol{u}}_i)$ is the nearest centroid's ID serving as the codeword of the subvector. Consequently, the complete principal components $\tilde{\boldsymbol{u}}$ are represented as a sequence of codewords, one for each subvector. The codeword length L_F relates to the number of centroids K via $L_F = \lceil log_2 K \rceil$ in a subspace. Flash samples a subset from the complete vector set to efficiently construct codebooks, following PQ and its variants [52, 67].

For each inserted vector $\boldsymbol{u}$, asymmetric distance tables (ADT) and codewords are simultaneously generated across M_F subspaces using precomputed codebooks, leveraging shared distance computations between each subvector $\tilde{\boldsymbol{u}}_i$ and the centroids in $\boldsymbol{C}_i$. This approach avoids duplicate calculations, enhancing efficiency. In the Candidate Acquisition (CA) stage, the ADT enables fast summation of partial distances to calculate distances between $\boldsymbol{u}$ and visited vertices. To adapt ADT for SIMD registers, Flash employs SQ to map each distance $dist$ in the table to discrete levels:

$$\eta(dist) = \left\lfloor \left(\frac{dist - dist_{\min}}{\Delta} \right) \cdot (2^H - 1) \right\rfloor \tag{9}$$

where $\eta(dist)$ is the quantized distance, $dist_{\min}$ is the minimum distance, Δ is the quantization step size ($\Delta = dist_{max} - dist_{min}$), and H is the bit number for a quantized distance value. With $K = 16$ and $H = 8$, each ADT is 128 bits in size. These ADTs are stored as *one-dimensional arrays* that can reside in registers during the insertion process, and are freed once the insertion is completed.

In the Neighbor Selection (NS) stage, distance computations among candidates preclude the use of ADT. To address this, Flash precomputes a symmetric distance table (SDT) that stores distances between centroids in each subspace. With M_F subspaces and K centroids, M_F SDTs are built, each holding K^2 distance values in a *two-dimensional array*. These values are quantized similarly to ADT for optimal caching. Flash obtains distances between candidates from SDTs based on their codewords. Note that SDTs are shared by all inserted vectors during index construction, eliminating numerous random memory accesses. To compare distance values from SDTs and ADTs for neighbor selection, Flash uses the same quantization step size Δ and target bit number H for both tables. It calculates the maximum and minimum distances ($dist^i_{max}$, $dist^i_{min}$) within each subspace, summing each $dist^i_{max}$ to obtain $dist_{max}$ and taking the lowest among $dist^i_{min}$ as $dist_{min}$.

3.3.4 ***Access-Aware Memory Layout.*** In the graph index, all vertices share a uniform neighbor list structure and size. A *contiguous memory block* is allocated for each vertex's vector and neighbor data, accessed via an offset determined by the vertex ID. For an inserted vector $\boldsymbol{u}$, the CA stage updates candidates via greedy search, visiting a vertex's neighbors in the sub-graph index and computing their distances to $\boldsymbol{u}$. Current graph indexes suffer from numerous random accesses when fetching neighbor vectors, as neighbor vectors are too big to be stored alongside neighbor IDs. By attaching neighbor codewords to these IDs, Flash computes distances directly in register-resident ADTs, avoiding random memory accesses. For each vertex, Flash stores the neighbor IDs in one fixed memory block, followed by the neighbor codewords in another (see Figure 5, lower right). To efficiently use SIMD instructions, Flash gathers B neighbor codewords in a subspace, processing them concurrently to ensure efficient SIMD register loading and parallel distance computations for B neighbors.

3.3.5 ***SIMD Acceleration.*** SIMD instructions support vectorized execution, enabling simultaneous processing of multiple data points. This can accelerate distance calculations between the inserted vector $\boldsymbol{u}$ and a batch of neighbors. Flash uses SIMD to look up ADTs and sum partial distances across subspaces. An ADT resides in a SIMD register, storing distances between $\boldsymbol{u}$ and centroids in each subspace. Flash employs shuffle operations to extract partial distances using neighbors' codewords. In a single operation, codewords for B neighbors are loaded into a register, which then serves as indices to access the partial distances in the ADT. Specifically, the shuffle operation rearranges the data within the register according to the indices, extracting partial distances from the ADT without complex conditional branching or excessive memory accesses. Partial distances from two subspaces are summed using SIMD instructions, and this process is repeated across all subspaces to compute the final distances for the B neighbors. By using SIMD lookups for ADTs, Flash minimizes random memory accesses and exploits high-speed registers for parallel arithmetic, thereby improving computational efficiency.

3.3.6 ***Implementation on HNSW.*** We integrate Flash into the HNSW algorithm to accelerate indexing and search processes. It begins by preprocessing the dataset to extract principal components (Section 3.3.2), then sampling a subset, dividing it into subspaces, generating codebooks, and pre-computing centroid distances for the SDT. During index construction (lines 2-8 of Algorithm 1), each vector $\boldsymbol{u}$ has its principal components partitioned by the codebooks. Distances between each subvector $\boldsymbol{u}_i$ and codebook centroids $\boldsymbol{C}_i$

form the ADT, and the nearest centroid ID is selected as the codeword for $\boldsymbol{u}_i$. The ADT is quantized (Section 3.3.3) and held in a register until insertion completes. For line 5, SIMD acceleration computes distances between $\boldsymbol{u}$ and batches of neighbors based on ADT (Section 3.3.5). For lines 6–7, distances between candidates are approximated via SDT lookups, using each candidate's codewords as indices. We tune the number of subspaces (M_F) and the dimensionality of principal components (d_F) to manage compression error (Theorem 1) without affecting `Flash`'s efficient memory access or SIMD operations. Other fixed parameters include bits per codeword (L_F) and batch size (B), set by the SIMD register size. The search procedure follows the CA paradigm, and we apply an additional reranking step based on original vectors to refine results.

Remarks. (1) Although many studies optimize the compression error of PQ, SQ, and PCA [29, 52, 57, 119], none are specifically tailored to meet the distinct requirements of graph indexing on modern CPU architectures. Integrating these compression methods into the graph construction process often yields limited efficiency gains or even degrades search performance, as they neither address random memory access patterns nor leverage SIMD efficiently. In contrast, `Flash` is a specialized compact coding strategy designed for graph index construction and optimized for current CPU architectures. Notably, `Flash` incorporates the complementary principles of established vector compression methods (e.g., PQ, SQ, PCA) within its framework and offers the flexibility to integrate new optimizations of these compression methods. While several studies integrate vector compression into search processes [29, 66, 103, 116], search differs markedly from index construction, which involves more complex tasks such as the NS stage. Moreover, it is crucial that compression methods avoid excessive overhead, maintaining a balance between construction speed and index quality. Recent work [66, 103, 113, 116] shows that boosting search performance often increases indexing time. (2) In `Flash`, codeword and ADT generations share identical distance computations (Section 3.3.3), prompting an integrated implementation to eliminate redundancies. We utilize the Eigen library for matrix manipulations throughout data processing, including principal component extraction, codebook generation, and distance table creation. When the maximum number of neighbors R exceeds the batch size B, we split each vertex's neighbor data into multiple blocks, each matching the batch size. This structure enables the efficient distance computation within each block using SIMD instructions.

3.3.7 ***Cost Analysis.*** In the CA stage, the time complexity of HNSW is approximately $O(R \cdot log(n))$, where R is the maximum number of neighbors and $log(n)$ denotes the search path length [75, 104]. For each hop, vector data of the visiting vertices' neighbors must be accessed. Therefore, the number of memory accesses in the original HNSW algorithm (NMA_{orig}) can be expressed as:

$$NMA_{orig} \sim O(R \cdot log(n)) \tag{10}$$

In our optimized implementation, neighbors' codewords and their respective IDs are stored contiguously, eliminating the need for random memory accesses to fetch neighbor vectors. As a result, the number of memory accesses in our implementation (NMA_{ours}) is:

$$NMA_{ours} \sim O(log(n)) \tag{11}$$

For simplicity, this analysis excludes cache hits. Notably, the `Flash` code is significantly smaller than the original vector data, resulting in a higher cache hit ratio. In the NS stage, computing distances between candidates requires accessing their vector data. For the original HNSW algorithm, the number of memory accesses is $O(C)$, where C denotes the size of the candidate set. In contrast, `Flash` maintains the entire SDT in the L1 cache, avoiding fetching vector data from main memory during the NS stage.

For each distance computation using SIMD acceleration, the number of register loads in the original HNSW (NRL_{orig}) is:

$$NRL_{orig} = \frac{32 \cdot D}{U} \tag{12}$$

where D is the vector dimensionality (each vector has D floating-point values) and U is the bit-width of a register. This is because each vector segment must be individually loaded into an SIMD register. Furthermore, complex arithmetic operations, including subtraction, multiplication, and addition, are required for distance computation. In contrast, the number of register loads in our implementation (NRL_{ours}) is

$$NRL_{ours} = \frac{M_F \cdot H}{U} \tag{13}$$

where M_F is the number of subspaces and H represents the number of bits to encode a partial distance within each subspace. In `Flash`, the ADT is register-resident. Thus, it loads the codewords of B neighbors within a subspace into a register in a single operation. To compute distances for these neighbors, M_F batches of codewords are loaded, corresponding to M_F register loads. The parameter B is typically set to U/H to align with register capacity. Additionally, the arithmetic operations in `Flash` are reduced to simple addition. With $D = 768$, $U = 128$, $M_F = 16$, and $H = 8$, the number of register loads required for a distance computation in the original HNSW is 192, whereas in our optimized version, it is reduced to just 1.

4 EVALUATION

We conduct a comprehensive experimental evaluation to answer the following research questions:

Q1: What impact do mainstream compact coding algorithms have on HNSW construction? (Sections 4.2 and 4.3)

Q2: How does `Flash` affect HNSW's construction efficiency, index size, and search performance? (Sections 4.2 and 4.3)

Q3: How does `Flash` scale across varying data volumes and segment counts (Section 4.4)?

Q4: How general is `Flash` with respect to various graph algorithms, HNSW optimizations, and SIMD instructions (Section 4.5)?

Q5: To what extent does `Flash` optimize memory accesses and arithmetic operations during index construction? (Section 4.6)

Q6: How do the parameters of `Flash` influence construction efficiency and index quality? (Section 4.7)

All source codes and datasets are publicly available at: https://github.com/ZJU-DAILY/HNSW-Flash.

4.1 Experimental Settings

4.1.1 ***Dataset.*** We use eight real-world vector datasets of diverse volumes and dimensions from deep embedding models, all of which are publicly accessible on Hugging Face and Big ANN Benchmarks